

5. Monitor 구성 정리

5. Monitor

5-1. 이 프로세스의 정체

Monitor는 네트워크를 통해 전달된 이벤트를 최종적으로 받아서 기록하고 시각화용 상태를 유지하는 프로세스다.

즉 역할은:

- 마지막 수신자
- 로그 저장소의 앞단
- 현재 시스템 상태를 보여주는 데이터 집계기

이다.

5-2. 왜 필요한가

이벤트가 잘 전달되었는지 확인하려면

최종 목적지가 있어야 한다.

Relay까지만 있으면 “중간까지 갔다”밖에 안 된다.

Monitor가 있어야:

- 실제 도달 성공 여부를 판단 가능
- 로그가 남음
- CPU spike / service down / recovery 같은 이벤트를 화면에 표현 가능
- retry / duplicate / failure 통계를 눈으로 확인 가능

즉 Monitor는 **결과 확인 지점**이다.

5-3. Monitor의 목적

목적은 5개다.

1. 최종 이벤트 수신

- Relay R2에서 전달된 메시지를 받는다.

2. 중복 처리

- retry 때문에 같은 event_id가 다시 오면 중복 저장 방지

3. 로그 저장

- 이벤트를 기록해서 나중에 분석 가능하게 함

4. 현재 상태 갱신

- host별 최신 상태를 유지

5. 시각화/통계용 데이터 제공

- Controller/UI나 dashboard가 읽을 수 있게 상태를 정리

5-4. 입력 / 출력

입력

A. R2에서 오는 EVENT

예:

```
{
  "event_id": "evt-host1-000041",
  "seq_no": 41,
  "host_id": "host-1",
  "agent_id": "agent-host-1",
  "type": "CPU_SPIKE",
  "severity": "WARN",
  "timestamp": "2026-03-19T10:00:13",
  "payload": {
    "cpu": 95
  }
}
```

B. Controller/UI의 제어 명령

예:

- reset logs
- clear dashboard
- pause monitor
- export logs

C. 내부 시간 정보

- timestamp 기록
- 통계 집계 주기

출력

A. R2로 보내는 ACK

- Monitor가 메시지를 정상 수신/기록했다는 응답

예:

```
{
  "msg_type": "ACK",
}
```

```
"ack_for": "evt-host1-000041",
"from": "monitor"
}
```

B. 로그 기록

- 파일 또는 메모리에 append

C. 최신 상태 데이터

- host별 current status map 갱신

D. 통계 데이터

- 총 수신 수
- 중복 수
- host별 장애 수
- 이벤트 타입별 빈도
- 전달 지연 측정 결과

E. UI용 조회 결과

- recent events
- current host status
- node summary

5-5. Monitor 내부 상태값

실제 구현 기준으로 아래 정도는 있어야 한다.

```
running_state
received_id_cache
event_log
host_state_table
event_count_by_type
event_count_by_host
duplicate_count
ack_enabled
max_recent_events
recent_events_buffer
last_seq_by_host
out_of_order_count
```

각 의미를 보자.

running_state

- **RUNNING**, **PAUSED**, **STOPPED**

received_id_cache

- 이미 기록한 event_id
- dedup 또는 idempotent 처리용

event_log

- 전체 이벤트 기록소
- 메모리 리스트, 파일, DB 중 하나

host_state_table

- host별 최신 상태 요약

예:

```
host-1:
  cpu_state = HIGH
  service_state = DOWN
  latency_state = HIGH
  last_event = SERVICE_DOWN
  updated_at = ...
```

event_count_by_type

- CPU_SPIKE 몇 건
- SERVICE_DOWN 몇 건 등

event_count_by_host

- host별 이벤트 수

duplicate_count

- 중복 이벤트 몇 개 버렸는지

ack_enabled

- 수신 후 ACK 보낼지 여부

max_recent_events

- 최근 이벤트 목록 최대 개수

recent_events_buffer

- 화면에 바로 보여줄 최근 이벤트들

last_seq_by_host

- host별 마지막 seq_no
- ordering 체크용

out_of_order_count

- 순서 꼬임 감지 횟수

5-6. Monitor가 담당하는 핵심 기능

기능 1. 최종 메시지 수신

목적

R2가 넘긴 EVENT를 받아 최종 처리 시작

작동 방식

- 이벤트 수신
- 기본 형식 검증
- dedup 검사
- 로그 기록 단계로 진행

실제 작동

```
R2 → Monitor  
Monitor: received evt-host1-000041
```

기능 2. 유효성 검증

목적

깨진 메시지나 필수 필드 누락 방지

검사 항목

- event_id
- seq_no
- host_id
- type
- timestamp

실제 작동

문제가 있으면:

- invalid log
- drop
- ACK 안 주거나 에러 응답 가능

초기 구현은 drop + log면 충분하다.

기능 3. dedup 또는 idempotent 처리

목적

같은 event_id가 다시 와도 로그 중복 저장 방지

왜 필요하나

R2가 Monitor ACK를 못 받으면 retry할 수 있다.

그러면 Monitor는 같은 메시지를 여러 번 볼 수 있다.

작동 방식

- `received_id_cache` 조회
- 이미 있으면 duplicate 판단
- 실제 로그 저장/상태 갱신은 생략
- ACK만 다시 보낼 수 있음

실제 작동

```
evt-host1-000041
이미 기록됨
→ duplicate
→ log append 생략
→ ACK 재전송 가능
```

기능 4. 이벤트 로그 저장

목적

이벤트 이력을 남겨 분석 가능하게 함

작동 방식

이벤트를 `event_log`에 append

형태는 간단히:

- 메모리 리스트
- 텍스트 파일
- JSON lines 파일

중 하나면 된다.

실제 작동 예

```
[2026-03-19 10:00:13] host-1 CPU_SPIKE seq=41 cpu=95
```

의미

이 로그는 나중에:

- 실험 결과 분석
 - retry/duplicate 관찰
 - 발표 시 evidence
로 쓸 수 있다.
-

기능 5. 최신 상태 갱신

목적

“지금 host 상태가 어떤가”를 빠르게 알 수 있게 함

로그만 쌓아서는 현재 상태를 바로 보기 어렵다.

그래서 Monitor는 event를 기반으로 **현재 상태 테이블**을 갱신해야 한다.

예시 규칙

- CPU_SPIKE 수신 → `host_state_table[host].cpu_state = HIGH`
- CPU_RECOVERED 수신 → `cpu_state = NORMAL`
- SERVICE_DOWN 수신 → `service_state = DOWN`
- SERVICE_RECOVERED 수신 → `service_state = UP`

실제 작동

이벤트를 “기록”하는 것과 “현재 상태 반영”하는 것은 별개 단계다.

기능 6. 최근 이벤트 목록 유지

목적

대시보드에서 최근 이벤트를 보여주기 위해

작동 방식

`recent_events_buffer`에 최근 N개 유지

예:

- 최근 20개 이벤트만 유지
- 새 이벤트가 오면 오래된 것 제거

실제 작동

UI는 이 버퍼를 읽어서 바로 화면에 표시 가능

기능 7. 통계 집계

목적

네트워크 동작 결과를 숫자로 보여주기 위해

최소 통계 항목

- total_received
- total_logged
- duplicate_count
- event_count_by_type
- event_count_by_host

있으면 좋은 항목

- average delivery delay
- out_of_order_count
- host별 active issue 수
- recent retry-related duplicate 수

실제 작동

이벤트가 올 때마다 카운터 증가

기능 8. ordering 체크

목적

이벤트 순서 꼬임 여부 확인

왜 필요하나

예:

- `SERVICE_RECOVERED` 가 `SERVICE_DOWN` 보다 먼저 보이면 이상함

작동 방식

host별 `last_seq_by_host` 를 유지

새 메시지가:

- 기존 seq보다 작거나
- 갑자기 역전되면

`out_of_order_count` 증가 또는 별도 표시

초기 구현 권장

순서를 강제로 재정렬하진 말고,

감지만 하는 수준으로 시작하는 게 적절하다.

기능 9. ACK 반환

목적

R2가 후단 전달 완료를 알 수 있게 함

작동 방식

메시지를 정상 처리했으면 ACK 반환

주의:

ACK 시점은 보통 다음 이후가 적절하다.

- 유효성 검사 통과
- dedup 또는 로그 처리 완료

즉 실제로 Monitor가 처리 완료했을 때 ACK를 보내야 한다.

기능 10. reset / clear

목적

실험 반복을 위해 로그와 상태 초기화

작동 방식

- event_log 비움
- host_state_table 초기화
- counters 초기화
- recent buffer 비움
- cache 초기화

5-7. Monitor의 실제 동작 흐름

정상 수신 흐름

1. R2가 EVENT 전송
2. Monitor 수신
3. 유효성 검사
4. dedup 검사
5. duplicate 아니면 log append
6. host_state_table 갱신
7. recent_events_buffer 갱신
8. 통계 갱신
9. R2에 ACK 반환

duplicate 수신 흐름

1. R2가 동일 event_id 재전송
2. Monitor 수신
3. received_id_cache 확인
4. 이미 처리된 메시지 확인
5. 로그 추가 안 함
6. 상태 갱신 안 함

7. ACK만 재전송 가능
8. duplicate_count 증가

ordering 이상 감지 흐름

1. host-1의 마지막 seq = 41
2. 새 이벤트 seq = 40 도착
3. out-of-order 판단
4. out_of_order_count 증가
5. 그래도 이벤트는 기록 가능
6. UI에 경고 표시 가능

reset 흐름

1. Controller/UI에서 reset 요청
2. Monitor가 로그 비움
3. 상태 테이블 초기화
4. 통계 초기화
5. recent event 목록 비움
6. 새 실험 준비 완료

5-8. Monitor가 가져야 하는 데이터 구조

실제 구현 감각을 위해 구체적으로 적으면 이렇다.

A. 이벤트 로그

예:

```
[
  {
    "event_id": "evt-host1-000041",
    "host_id": "host-1",
    "seq_no": 41,
    "type": "CPU_SPIKE",
    "timestamp": "2026-03-19T10:00:13",
    "payload": { "cpu": 95 }
  }
]
```

B. host_state_table

예:

```
{
  "host-1": {
    "cpu_state": "HIGH",
    "memory_state": "NORMAL",
    "service_state": "DOWN",
    "latency_state": "HIGH",
    "last_event_type": "SERVICE_DOWN",
    "last_seq_no": 44,
    "updated_at": "2026-03-19T10:00:20"
  }
}
```

C. 최근 이벤트 버퍼

예:

```
[
  "evt-host1-000044 SERVICE_DOWN",
  "evt-host1-000043 LATENCY_HIGH",
  "evt-host1-000042 CPU_SPIKE"
]
```

D. 통계

예:

```
{
  "total_received": 120,
  "total_logged": 108,
  "duplicate_count": 12,
  "out_of_order_count": 2,
  "event_count_by_type": {
    "CPU_SPIKE": 30,
    "SERVICE_DOWN": 12,
    "SERVICE_RECOVERED": 10
  }
}
```

5-9. Monitor가 해석해야 하는 이벤트 예

이벤트 타입별로 어떤 상태를 바꾸는지 정리한다.

CPU_SPIKE

- cpu_state = HIGH

CPU_RECOVERED

- cpu_state = NORMAL

MEMORY_PRESSURE

- memory_state = HIGH

SERVICE_DOWN

- service_state = DOWN

SERVICE_RECOVERED

- service_state = UP

LATENCY_HIGH

- latency_state = HIGH

LATENCY_NORMALIZED

- latency_state = NORMAL

즉 Monitor는 단순 저장기가 아니라
이벤트를 해석해서 현재 상태를 만든다.

5-10. Monitor가 보여줄 수 있는 것

이건 나중에 UI와 연결된다.

최소 표시 항목

- 최근 이벤트 목록
- host별 현재 상태
- 총 이벤트 수
- 중복 drop 수
- out-of-order 수

있으면 좋은 것

- event type별 그래프
- 시간대별 이벤트 수
- node status
- relay retry 횟수

하지만 Monitor 자체는 “그리는 역할”보다
보여줄 데이터를 준비하는 역할이라고 이해하는 게 맞다.

5-11. Monitor가 하지 않는 것

중요하다. Monitor는 아래를 하지 않는다.

- Host 상태 직접 감지 안 함
- 이벤트 생성 안 함
- relay처럼 후단 forward 안 함
- retry 정책 중심 처리 안 함
- 사용자가 직접 누르는 UI 자체는 아님

즉 Monitor는 **최종 수신/기록/집계 프로세스**다.

5-12. Monitor만 따로 구조도로 그리면

Monitor
입력
- EVENT from R2
- Control from Controller/UI
내부 상태
- received_id_cache
- event_log
- host_state_table
- recent_events_buffer
- event_count_by_type
- last_seq_by_host
내부 기능
- receive
- validate
- dedup / idempotent check
- log append
- update current host state
- update statistics
- detect out-of-order
- send ACK to R2
출력
- ACK to R2
- logs
- dashboard data
- statistics

5-13. 앞단과 연결해서 보면

```
Host Simulator
  ↓
Local Agent
  ↓
Relay R1
  ↓
Relay R2
  ↓
Monitor
```

여기서 Monitor는:

- 최종 sink
- 로그 저장소
- 상태 집계기

다.

5-14. 핵심 한 문장 정리

Monitor는 Relay R2가 전달한 이벤트를 최종 수신하여 중복 여부를 검사하고, 이벤트 로그와 현재 host 상태를 갱신하며, 통계와 대시보드용 데이터를 생성한 뒤 ACK를 반환하는 프로세스다.

5-15. 지금까지 다섯 프로세스 흐름 요약

```
Host Simulator
- 상태/장애 생성

↓ 상태 조회

Local Agent
- polling
- 이벤트 생성
- R1 publish

↓ EVENT

Relay R1
- 수신
- dedup
- R2 forward
- ACK wait / retry

↓ EVENT
```

Relay R2

- 수신
- dedup
- Monitor forward
- downstream ACK wait / retry / buffer

↓ EVENT

Monitor

- 최종 수신
- dedup
- 로그 저장
- 현재 상태 갱신
- 통계 생성
- ACK 반환

다음